Text-to-SQL on WikiSQL: how my models work

Hejer Ben Amri | Natural Language Processing project | November 2026

1 The task and the general idea

A user asks a question about a table, for example "What is the capital of Peru?", and the system must write an SQL query that answers it. I treat this as *text generation*: the model reads a sequence of tokens and writes another sequence of tokens. All my models are **encoder–decoder** networks. The **encoder** reads the input and builds a representation of it. The **decoder** uses this representation to write the query, one token at a time. Everything is written from scratch in PyTorch, without a Transformer and without any pretrained model.

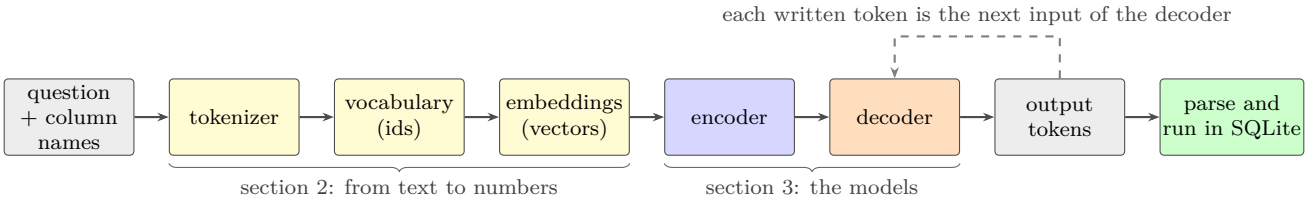


Figure 1: The whole system. The preprocessing is shared by all models; only the encoder and the decoder change from one model to the next.

2 From text to numbers

Input and output format. A network reads only numbers, so I first decide how a question and a table become one sequence. The input is the question, a separator, and then every column name after a column marker. The output is the query, written with special tokens for its structure (select, aggregator, where, operator, and). Values are written as words of the question, and at the end I recover the exact characters from the original question. Because the output is a sequence, I can turn it back into a query and run it in a real database (Figure 2). Figure 1 shows the whole path.

Question: *What is the capital of Peru?* **Table columns:** Country, Capital, Population

Input: `what` `is` `the` `capital` `of` `peru` `?` `<sep>` `<col>` `country` `<col>` `capital` `<col>` `population`

Output: `<sel>` `<agg_none>` `capital` `<where>` `country` `<eq>` `peru` `<eos>`

SQL: `SELECT Capital FROM table WHERE Country = 'peru'`

Figure 2: One example: how a question and a table become the input and the output sequences. Orange: structure tokens, blue: column names.

Tokenization. I normalize the text (Unicode form NFKC, lowercase) and cut it with a regular expression into numbers ("3.5" stays whole), words, and single punctuation marks. Every token keeps its position in the original text; this is what allows me to recover the exact value of a condition.

Vocabulary. The vocabulary is a table that gives an integer id to each token. I build it *only from the training set*, and I keep the words that appear at least twice, so that nothing from the dev or test set leaks in and very rare words do not add noise. It also holds the special tokens: padding, unknown word, start and end of output, and the structure tokens. Every other word becomes `<unk>`. This is a weakness: *a word never seen in training cannot be written.* It is the reason for the last two models.

Batching and padding. Sequences have different lengths, so I pad the short ones with `<pad>` to form a batch. The real lengths are given to the LSTM and to the attention, so that the padding is ignored. I also group questions of similar length in the same batch, which wastes much less computation on padding.

Embeddings. Each token id is mapped to a learned vector of 256 numbers (an embedding table). Words used in similar ways get similar vectors. The encoder and the decoder each have their own table, with dropout on the embeddings.

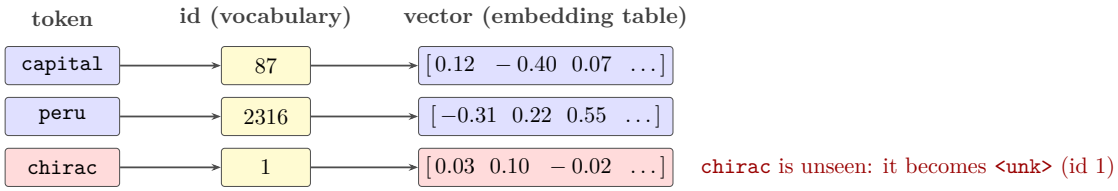


Figure 3: From a token to a vector (illustrative values). A word never seen in training gets the id of `<unk>`; then its own letters are lost, which is the weakness that M3 and M2-BPE address.

3 The models, and what each one improves

All models use the same data, the same sizes and the same training: the cross-entropy loss, and *teacher forcing* (during training the decoder receives the true previous token, not its own guess), optimized with AdamW. I also clip the gradients, reduce the learning rate when the loss on the dev set stops improving, and keep the best epoch on dev. This way a difference between two models comes only from the component I change. Each model answers a weakness of the one before.

M0, the baseline. It never reads the question: it always writes the most frequent query shape of the training set. It shows what a model gets without understanding anything, and so what the others must beat.

3.1 M1: LSTM encoder–decoder

An LSTM is a recurrent network with a memory cell protected by three gates (forget, input and output), which decide what to erase, what to write and what to show. At each step, with input x_t and previous states h_{t-1}, c_{t-1}:

$$f_t = \sigma(W_f[x_t; h_{t-1}] + b_f), \quad i_t = \sigma(W_i[x_t; h_{t-1}] + b_i), \quad o_t = \sigma(W_o[x_t; h_{t-1}] + b_o),$$
$$c_t = f_t \odot c_{t-1} + i_t \odot \tanh(W_c[x_t; h_{t-1}] + b_c), \quad h_t = o_t \odot \tanh(c_t).$$

I wrote the decoder's LSTM cell myself from these equations and tested it against PyTorch's; the encoder uses PyTorch's LSTM layer. In M1 the encoder reads the input from left to right. Its *final state* summarizes the whole input and becomes the initial state of the decoder, which writes the query and scores every vocabulary word with a softmax at each step. *Weaknesses:* the whole question and all the column names are squeezed into **one vector of fixed size**, and the state of a word knows only the words on its left.

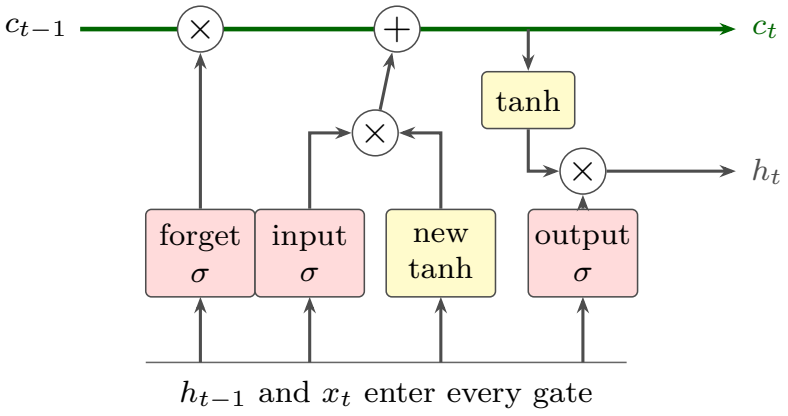
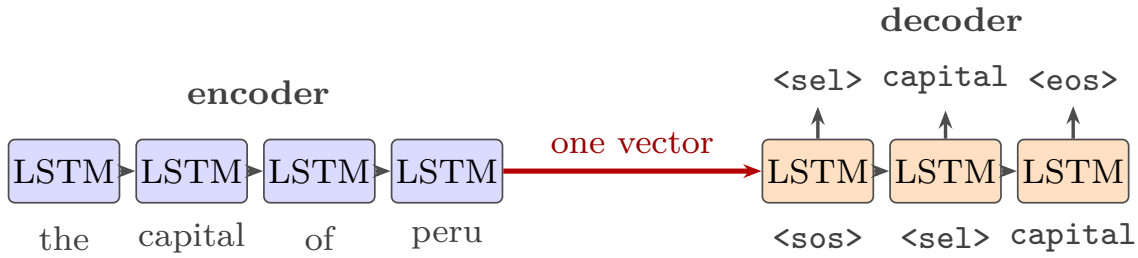


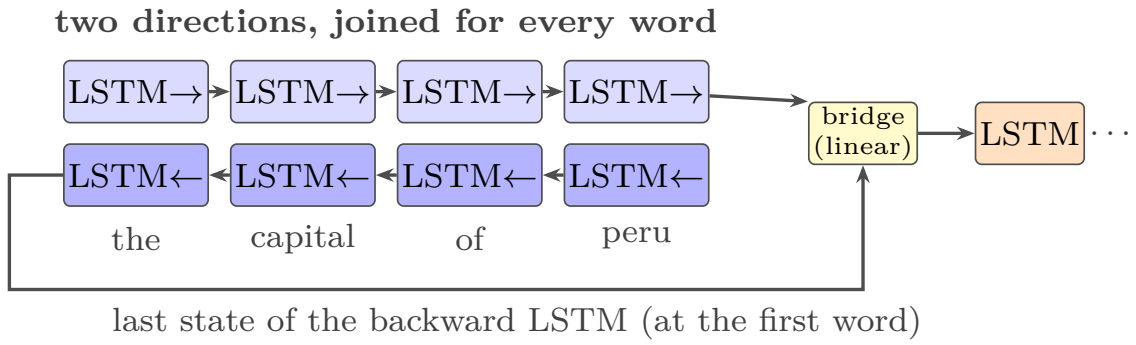
Figure 4: An LSTM cell. The memory c runs along the top and is changed only by the gates: forget (erase), input (write) and output (show).

3.2 M1b: bidirectional encoder and bridge

Here the encoder has two LSTMs that read the input in opposite directions, and their states are joined for every word, so a word is described by its left *and* right context. The final states of the two directions are joined and mapped by a linear layer (the *bridge*) to the size of the decoder. This change makes the description of each word richer. It does not remove the bottleneck, because the decoder still receives a single vector; its real value is to give the next model better states to look at.



(a) M1: one final state for the decoder



(b) M1b: bidirectional encoder and bridge

Figure 5: Models without attention. In M1 the decoder only receives the last state of the encoder.

3.3 M2: attention

M2 removes the bottleneck. The decoder keeps *all* the encoder states h_i and, at every output step, looks at them again. With the decoder state s_t, it computes a score for every input position (additive attention), turns the scores into weights with a softmax, and reads a weighted average of the states:

$$\text{score}_i = v^\top \tanh(W_h h_i + W_s s_t), \qquad \alpha = \text{softmax}(\text{score}), \qquad c_t = \sum_i \alpha_i h_i, \qquad \tilde{s}_t = \tanh(W_c[s_t; c_t]).$$

The state $\tilde{s}_t$ is then used to score the next token. Intuitively, when the model writes a column name it looks at the column names in the input, and when it writes a value it looks at the entity in the question. This solves the bottleneck and the alignment between output words and input words. *Weakness:* the model still writes only words of its vocabulary.

3.4 M3: copy mechanism (pointer-generator)

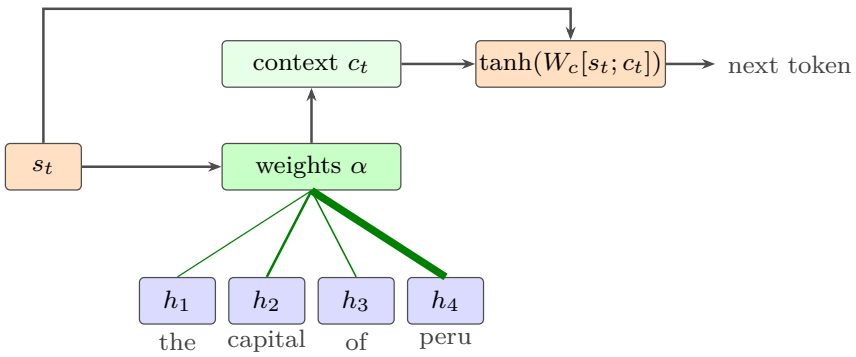
Many questions contain names of people and places that never appear in training, so they are `<unk>`. M3 adds a second way to write a word. At each step it either **generates** a word of the vocabulary (probability p_{gen}) or **copies** a word of the input (probability $1 - p_{gen}$). The probability of copying a position is simply its attention weight, so the attention that tells where to look also tells what to copy:

$$P(w) = p_{gen}\, P_{\text{vocab}}(w) + (1 - p_{gen}) \sum_{i:\, x_i = w} \alpha_i, \qquad p_{gen} = \sigma(W_g[s_t; c_t; \text{emb}(y_{t-1})]).$$

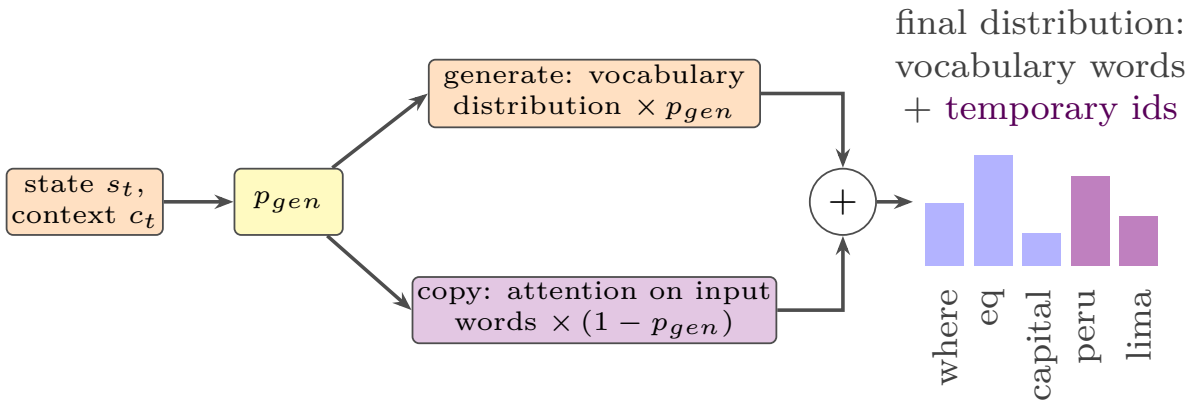
An unknown word of the input gets a temporary id ($V, V{+}1, \dots$) that is valid only for that example, so the model can learn to write it by copying. The gate learns to be close to 1 for structure tokens (`<where>`, `<eq>`) and close to 0 for words to copy. This costs only one small extra layer. *Weakness:* the vocabulary is still large, and a copied word must appear in the input.

3.5 M2-BPE: attention with byte-pair encoding

BPE answers the same problem as M3 in a different way: it changes the *units* instead of adding a copy gate. I learn the pieces from the training words. I start with every word cut into characters, then I repeatedly merge the most frequent pair of neighbouring pieces into one new piece; each merge adds one entry to the vocabulary. Frequent words become one piece, rare words a few pieces, and any new word can still be written, in the worst case character by character, so there is no unknown word.



(c) M2: the decoder looks at all input states



(d) M3: generate or copy

Figure 6: Attention (M2) and the copy mechanism (M3). In (c), a thicker line means a larger weight α_i.



The model is the attention model M2, unchanged. The vocabulary is much smaller, so the embedding and output tables are smaller too. The price is longer sequences, and the model must spell each name piece by piece; a small spelling error gives a wrong name.

4 Decoding and summary

At test time the decoder starts from `<sos>` and writes without help: **greedy decoding** takes the best token at each step until `<eos>`, and **beam search** keeps the k best partial queries. A query is then parsed and executed in SQLite.

Model	What it adds	Problem it addresses	Problem that remains
M0	most frequent query	(baseline, no learning)	does not read the question
M1	LSTM encoder and decoder	learns to read and write	one vector for everything; left context only
M1b	two directions + bridge	richer state for each word	still one vector for the decoder
M2	attention over all states	the bottleneck, alignment	only vocabulary words
M3	copy gate p_{gen}	unknown words of the input	large vocabulary
M2-BPE	subword pieces	unknown words, vocabulary size	longer sequences, spelling errors

Table 1: Each model fixes a weakness of the previous one.

How the models fit together. The six models are a ladder, and I built them in this order on purpose. M1 shows that an encoder–decoder can learn the task at all. M1b and M2 improve how the input is *read* and *used*: first a better description of each word, then the freedom to look back at any word at any step. M3 and M2-BPE improve how the output is *written*: they remove the limit of a fixed vocabulary, one by copying and the other by spelling with pieces. Because I changed one thing at a time, I can say which idea solves which problem.